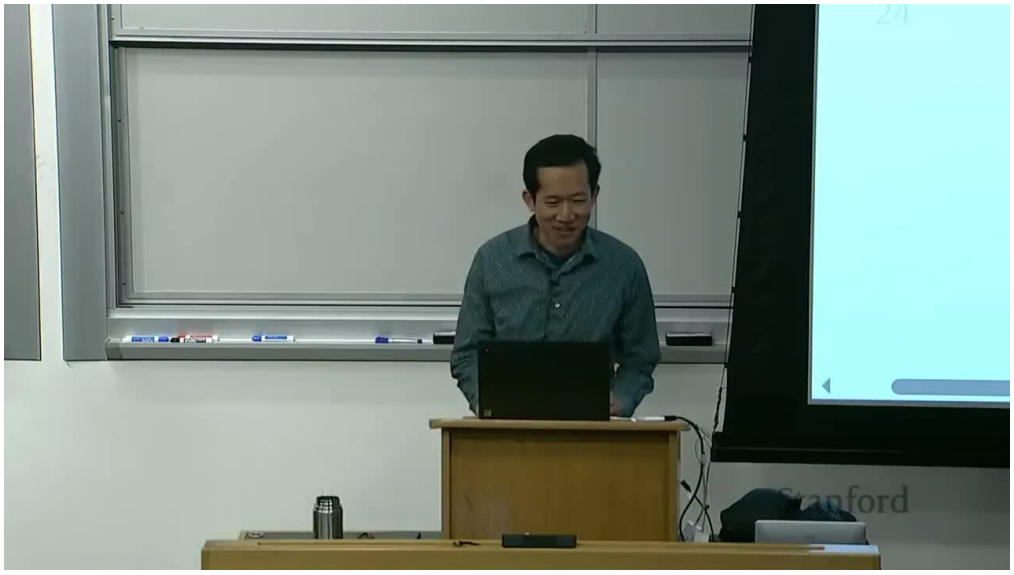
课程笔记

CS336 第 1 讲：课程导论、效率世界观与 Tokenization

Codex 基于 Stanford CS336 2025 第 1 讲视频、字幕与官方课程材料重写

2026-04-07



视频作者/频道：Stanford Online
发布日期：2025 年 04 月 24 日
视频时长：1:18:59
视频链接：`https://www.youtube.com/watch?v=SQ3fZ1sAqXI`

目录

1 为什么要在 2025 年还讲一门“从零构建语言模型”的课 2

1.1 开场不是寒暄，而是在公开这门课的立场 2
1.2 真正的问题：研究者与底层技术栈正在脱节 2
1.3 工业化让“理解”变得比“调用”更难 3
1.4 小模型不等于大模型，但仍然有三类知识值得迁移 4
1.5 为什么“More is different”不是一句口号 4
1.6 直觉并不总能被理论解释：SwiGLU 与 bitter lesson 5
1.7 本章小结 6

2 效率世界观：这门课真正训练的是做设计决策的能力 6

2.1 课程并不把效率视为一个后处理问题 6
2.2 设计决策地图：为什么 CS336 不是若干专题拼盘 7
2.3 后续课程地图：17 讲在整体上分别负责什么 7
2.4 为什么这门课会非常重：因为它把研究工程肌肉当成主要训练目标 8
2.5 本章小结 8

3 历史演化与开放性格局：这门课站在什么时间点上 8

3.1 从 Shannon 到 Transformer：语言建模问题是怎样形成的 8
3.2 为什么“扩规模”会改写整个研究范式 9
3.3 开放性不是二元变量，而是一条连续谱 9
3.4 “今天的 frontier models”是课程时点的快照，而不是永恒结论 10
3.5 本章小结 10

4 Tokenization：整条流水线里的第一个具体设计决策 10

4.1 Tokenizer 的职责：在字符串与离散序列之间搭桥 10
4.2 为什么这门课从 BPE 开始，而不是直接追求 tokenizer-free 的优雅 11
4.3 视频里的 tokenization 段落，其实已经在预演后面整门课 11
4.4 Tokenization 不是孤立预处理，它会改变后续所有模块 12
4.5 从第一讲到第二讲：为什么下一步自然是 PyTorch 与资源核算 12
4.6 本章小结 13

5 总结与延伸 13

1 为什么要在 2025 年还讲一门“从零构建语言模型”的课

第一讲一开始并没有急着进入 Transformer、tokenizer 或优化器，而是先回答一个更根本的问题：**在一个大家都可以直接调用现成大模型 API 的时代，为什么还要回过头来，从最底层重新理解语言模型系统？**这不是一个课程营销问题，而是整门 CS336 的哲学起点。

课程给出的回答很明确：今天的研究者与工程师正在迅速上移到更高抽象层，生产力当然提高了，但也更容易与真实技术栈脱节。你可以只靠 prompt、少量微调，甚至现成服务，就做出很多可用系统；可一旦你要回答“为什么这个模型在这里失败”“到底是数据、架构、系统、规模还是后训练在限制结果”“应该优先优化哪一层”这些更本质的问题，单纯停留在 API 层往往就不够了。

1.1 开场不是寒暄，而是在公开这门课的立场

开场部分里，讲者先介绍课程团队，随后马上点出两件事：这是 CS336 的第二次开课，规模比上一年大了约 50%；同时，课程会把讲义和视频公开到 YouTube，让更广泛的人也能跟着学习。这个安排本身就传递了一个信号：**它不想只服务于校内课堂，而是把“如何系统理解语言模型”当成一个值得公开传播的工程教育问题**。

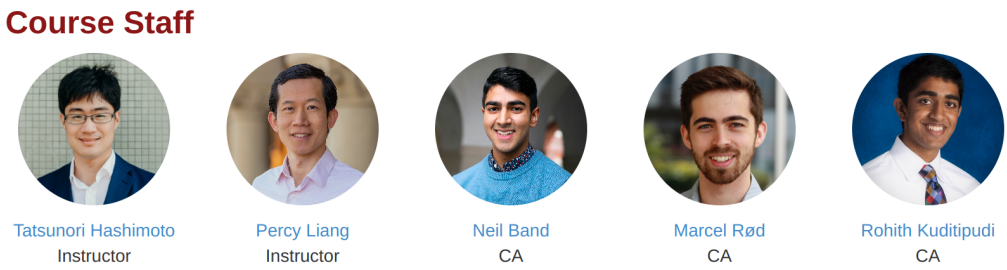


图 1: 课程团队亮相。第一讲在教学上既是导论，也是对整门课能力模型的公开说明。

和很多“热点技术导论课”不同，这门课的气质从一开始就偏工程研究训练。它不是承诺带你快速掌握一串前沿名词，而是强调：如果你真想进入这一领域并具备判断力，就得愿意去理解模型背后的全部链条，包括数据、tokenization、架构、训练、并行、推理、评估与对齐。

1.2 真正的问题：研究者与底层技术栈正在脱节

讲者用一个很有力量的三段式对比来解释这个危机：

- 更早的时候，研究者会亲手实现并训练自己的模型。
- 再后来，研究者会下载像 BERT 这样的公开模型并做下游微调。
- 到今天，很多人只需要围绕专有模型做 prompt 和工作流封装，就可以完成大量任务。

课程并没有简单地批评这种趋势，因为抽象层变高本来就会带来更高生产力，很多有价值的应用研究也正是因为这种抽象才成为可能。真正的问题在于：**这些抽象不是像编程语言或操作系统那样相对稳定、边界清晰的抽象；它们是“会漏水”的**。当抽象一旦泄漏，系统背后的训练数据、上下文窗口、KV cache、并行策略、优化目标、对齐方式等因素都会重新变得重要。

第一讲的基本判断

真正高水平的语言模型研究与工程能力，不只是“能不能把一个模型调起来”，而是能不能判断一个行为结果究竟是由哪一层机制决定的，并据此做出正确的设计与优化决策。

所以，课程选择了一条很笨但也很扎实的路：**understanding via building**。不是只讲结论，而是把整条流水线重新搭一遍，让学生在构建过程中逐步获得判断力。

1.3 工业化让“理解”变得比“调用”更难

可问题马上就出现了：前沿模型已经极度工业化。课程列举的数字本身就足够说明现实的门槛之高：GPT-4 常被传闻拥有约 1.8T 参数、训练成本达到亿美元量级；xAI 在建超大 GPU 集群；Stargate 这类项目讨论的是数千亿美元规模的基础设施投入。无论具体数字是否完全准确，方向都很清楚：**最顶层模型训练已经不是普通研究者可以完整复现的活动**。

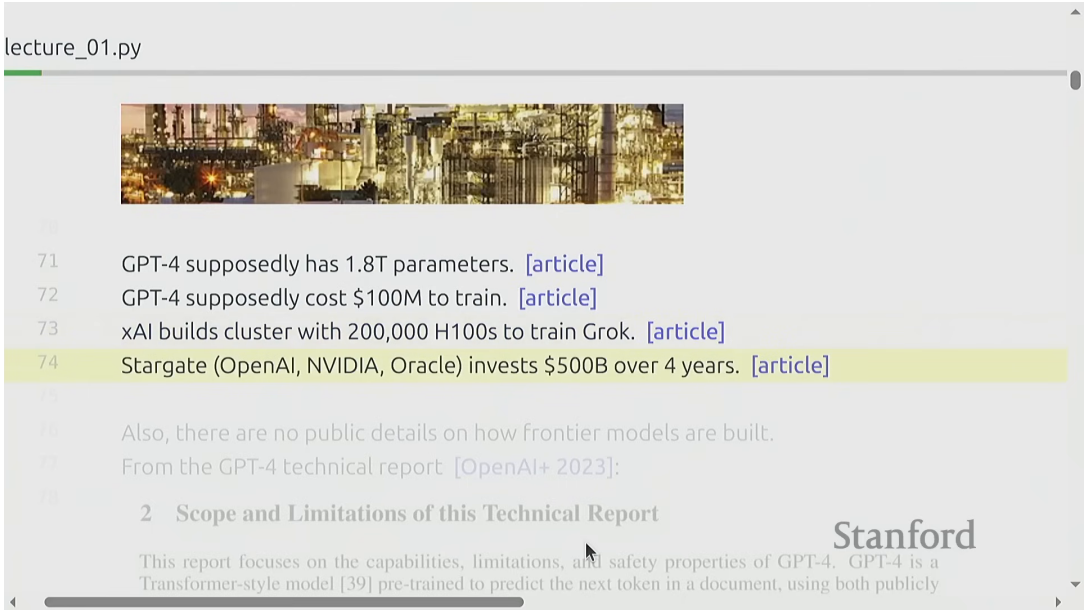


图 2: 课程视频里直接展示 `lecture_01.py` 的执行界面，把“工业化”“GPT-4 细节缺失”等讨论放在可执行讲义里展开。[1]

更棘手的是，最关键的系统又未必公开足够细节。课程专门引用 GPT-4 technical report 作为例子：即使论文存在，也会因为竞争压力与安全约束而刻意略去大部分核心工程细节。于是学习者面临一个非常现实的悖论：**越重要的系统，越难通过“完整复现论文”来理解**。

[1] 视频画面时间区间：00:04:11–00:04:55。该图来自课程视频中的屏幕画面。

2 Scope and Limitations of this Technical Report

This report focuses on the capabilities, limitations, and safety properties of GPT-4. GPT-4 is a Transformer-style model [39] pre-trained to predict the next token in a document, using both publicly available data (such as internet data) and data licensed from third-party providers. The model was then fine-tuned using Reinforcement Learning from Human Feedback (RLHF) [40]. Given both the competitive landscape and the safety implications of large-scale models like GPT-4, this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar.

We are committed to independent auditing of our technologies, and shared some initial steps and ideas in this area in the system card accompanying this release.[2] We plan to make further technical details available to additional third parties who can advise us on how to weigh the competitive and safety considerations above against the scientific value of further transparency.

图 3: GPT-4 technical report 中对关键工程细节的克制公开，正是这门课存在的重要背景。

这就是第一讲真正想解决的矛盾：既然 frontier model 训练离我们很远，为什么“从零理解”仍然值得？课程的答案不是假装小模型等价于大模型，而是精确地区分：哪些知识能迁移，哪些不能。

1.4 小模型不等于大模型，但仍然有三类知识值得迁移

第一讲最有价值的概念之一，就是它把“可迁移的收获”分成了三类：

类别	核心问题	课程中的典型内容
Mechanics	东西是怎么工作的	Transformer、注意力、并行训练、KV cache、优化器状态
Mindset	应该用什么方式思考	资源核算、效率优先、roofline、scaling laws、通信代价
Intuitions	哪些设计在经验上更有效	激活函数、数据配比、MoE、不同训练 recipe 的经验判断

表 1: 第一讲区分了三种知识：机制、思维方式和经验直觉。

课程的立场非常冷静：Mechanics 和 Mindset 是比较稳的迁移对象，因为无论模型规模怎么变，系统终究要处理张量、显存、带宽、并行、优化与推理问题；而 Intuitions 只能部分迁移，因为很多看似正确的经验，会随着规模、数据分布和硬件条件改变而失效。

为什么 Mindset 往往比背结论更重要

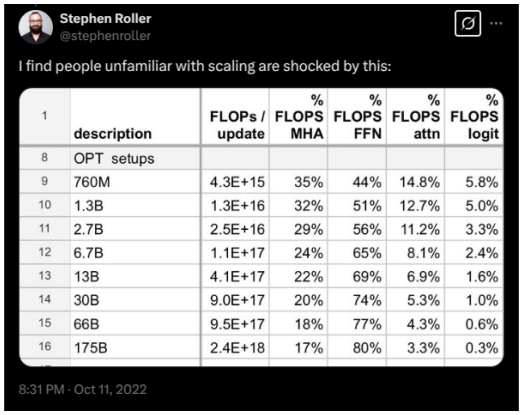
很多系统组件其实不是“现在才被发明”，而是早就存在。真正推动语言模型进入新阶段的，往往不是某一个单点技巧，而是把规模、效率、数据和硬件约束一起纳入思考的方式。这就是课程所说的 mindset。

1.5 为什么“More is different”不是一句口号

课程接着解释，为什么小模型世界里的观察不能被直接外推到大模型。它给了两个非常典型的例子。

第一个例子是不同规模下 attention 与 MLP 的 FLOPs 占比。小模型里，二者的计算占比看起来可能差不多；但到 GPT-3 级别以后，MLP 的计算占比明显更重。如果你长期停留在小模型上优化 attention，却把 MLP 视为次要项，那么当规模扩大后，你可能会发现自己优化错了对象。

第二个例子是行为涌现。某些能力在小模型区间里看起来几乎不存在，但当训练 FLOPs 或参数规模跨过某个阈值后，性能曲线会突然变得完全不同。于是，在低规模区间里得出的“这个方向没用”结论，可能只是因为你还没走到真正关键的规模区域。



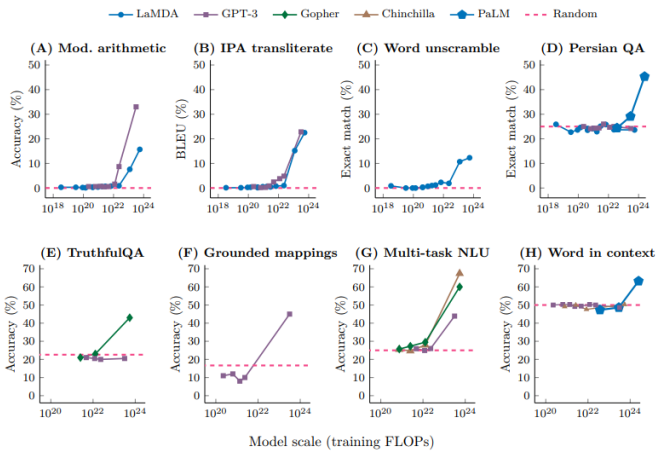
Stephen Roller
@stephenroller

I find people unfamiliar with scaling are shocked by this:

1	description	FLOPs / update	% FLOPS MHA	% FLOPS FFN	% FLOPS attn	% FLOPS logit
8	OPT setups					
9	760M	4.3E+15	35%	44%	14.8%	5.8%
10	1.3B	1.3E+16	32%	51%	12.7%	5.0%
11	2.7B	2.5E+16	29%	56%	11.2%	3.3%
12	6.7B	1.1E+17	24%	65%	8.1%	2.4%
13	13B	4.1E+17	22%	69%	6.9%	1.6%
14	30B	9.0E+17	20%	74%	5.3%	1.0%
15	66B	9.5E+17	18%	77%	4.3%	0.6%
16	175B	2.4E+18	17%	80%	3.3%	0.3%

8:31 PM · Oct 11, 2022

(a) FLOPs 结构会随规模变化



(b) 能力曲线可能出现明显阈值效应

图 4: “More is different”的意思不是盲目迷信大模型，而是承认很多系统规律本身就会随规模发生相变。

第一讲对小模型实验的边界提醒

小模型是很好的受控实验台，但它不是 frontier 模型的缩小镜像。拿小模型做实验的价值，在于学机制、练方法、建成本意识，而不是从小规模结果里机械外推大规模世界的全部性质。

1.6 直觉并不总能被理论解释：SwiGLU 与 bitter lesson

课程还用 Noam Shazeer 关于 SwiGLU 的例子提醒学生：很多设计之所以被广泛采用，并不是因为理论已经完全解释了它，而是因为实验表现就是更好。论文里那句近乎自嘲的“divine benevolence”，本质上是在说：**现代模型设计里仍然有很多经验主义成分**。

4 Conclusions

We have extended the GLU family of layers and proposed their use in Transformer. In a transfer-learning setup, the new variants seem to produce better perplexities for the de-noising objective used in pre-training, as well as better results on many downstream language-understanding tasks. These architectures are simple to implement, and have no apparent computational drawbacks. We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.

图 5: SwiGLU 例子说明：在现代大模型里，很多有效设计最初并不是靠优雅理论赢下来的，而是先靠经验表现。

接着课程把 bitter lesson 重新落回一个更工程化的命题：真正重要的，不是“算法不重要”，而是**能够随规模一起增长的算法才重要**。这也是为什么课程迅速把讨论压缩成一句更适合工程脑袋的话：

$$\text{accuracy} \approx \text{efficiency} \times \text{resources}$$

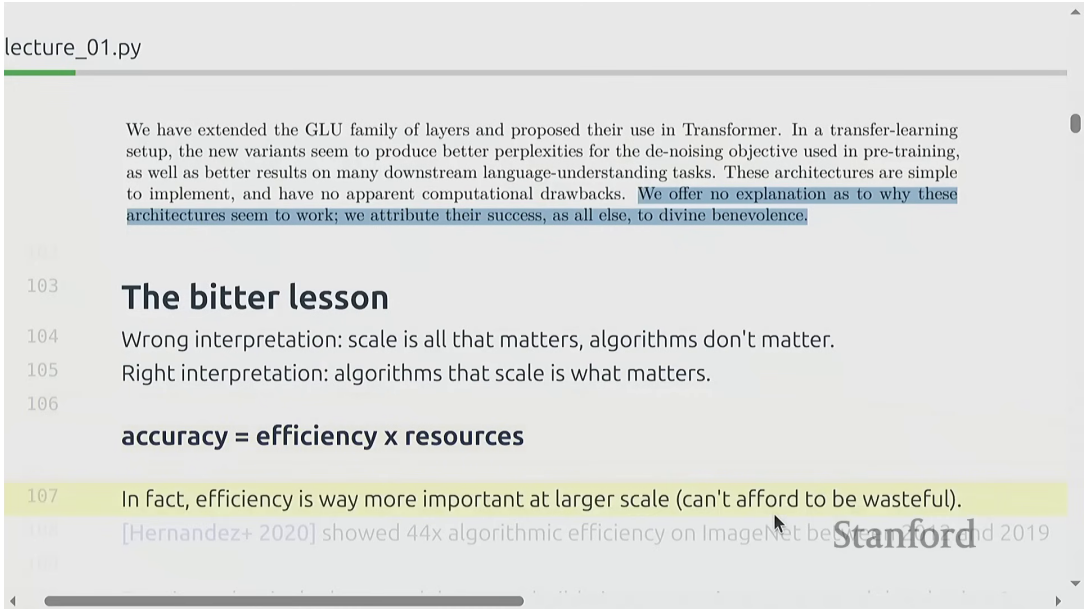


图 6: 课程视频中的屏幕画面把 bitter lesson 与“accuracy = efficiency x resources”放在同一页上，突出这门课的效率世界观。[2]

这句话非常值得细读。它不是严格数学定理，而是一种设计世界观：如果资源固定，那么你就必须追求效率；如果效率提升，那么同样资源就能换来更高模型质量；而在大模型时代，系统性浪费会被成倍放大，所以效率不再是“工程锦上添花”，而是算法设计本身的一部分。

1.7 本章小结

第一讲前半段完成了整门课最重要的哲学铺垫。它说明了为什么今天仍然需要“从零构建”的训练路径，也说明了这条路不等于天真地复现 frontier model，而是要有意识地分辨什么是可迁移的核心能力。工业化确实让前沿系统离普通研究者更远，但也正因如此，Mechanics 与 Mindset 的训练才更不可或缺。

2 效率世界观：这门课真正训练的是做设计决策的能力

2.1 课程并不把效率视为一个后处理问题

很多人会把“效率”理解成模型训练完成之后的优化阶段，比如做一做 kernel fusion、量化、服务化部署；但第一讲实际上从一开始就把效率抬到了更高位置。课程反复问的问题是：**在固定的数据和硬件预算下，你到底应该训练什么样的模型？**

一旦这样提问，很多看似独立的问题就会自动连接起来。数据处理不是单独的话题，因为坏数据会浪费训练预算；tokenization 不是单独的话题，因为它会改变序列长度与后续计算成本；架构选择不是单独的话题，因为不同架构的参数利用率、带宽压力和并行友好性不同；后训练也不是独立附加层，因为它可能改变你需要多大 base model 才能达到可用效果。

[2] 视频画面时间区间：00:09:56–00:10:12。该图来自课程视频中的屏幕画面。

这门课的统一提问方式

不要先问“某个技术炫不炫”，而要先问：在给定资源、目标能力和系统约束下，它到底能不能让整条流水线更优。

2.2 设计决策地图：为什么 CS336 不是若干专题拼盘

第一讲最关键的一张图，是把语言模型构建看成一连串设计决策，而不是一堆孤立技巧。课程真正关心的是整条 pipeline 的联动关系。

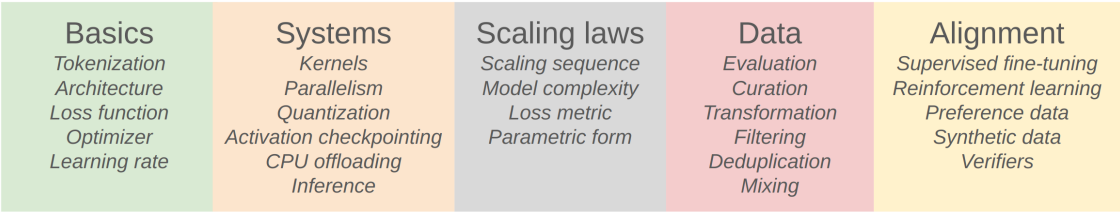


图 7: 从数据、tokenization、架构、训练到系统与对齐，语言模型是一条连续设计链，而不是一组彼此无关的 trick。

这张图的价值在于，它逼着读者看到“设计的耦合性”。例如：

- tokenization 会影响序列长度，从而影响 attention FLOPs、KV cache 大小和吞吐。
- 架构会影响并行方式、访存模式和可用 kernel 优化空间。
- 训练 recipe 会影响 scaling law 外推是否可靠。
- 评估与数据处理会反过来决定你到底应该优化什么能力。
- 对齐与 RLVR 会影响你对 base model、推理成本和反馈信号的要求。

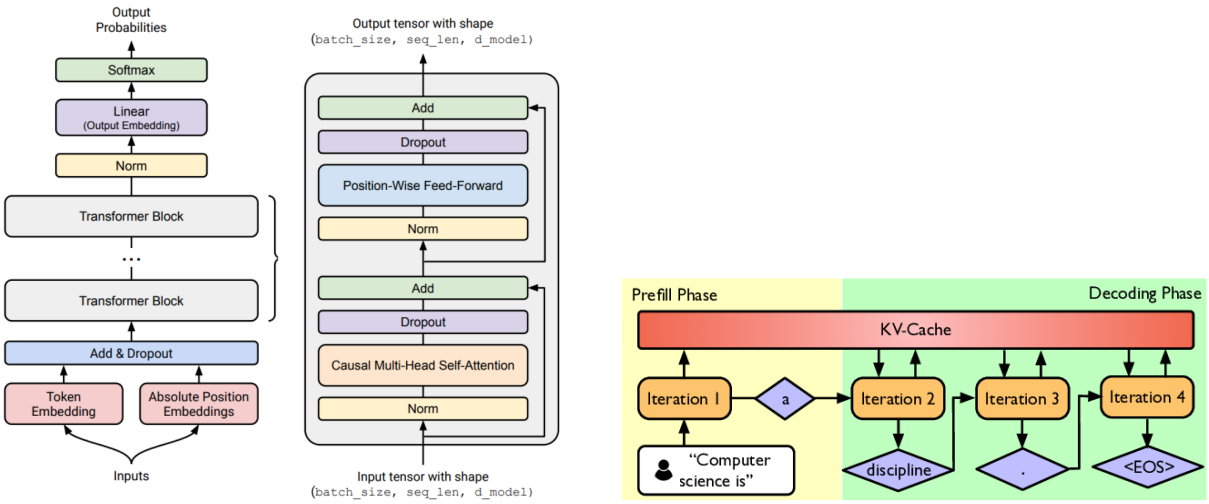
所以，从第一讲开始，课程就在训练一种非常具体的能力：**把每个局部选择放回整条系统链条里评估**。

2.3 后续课程地图：17 讲在整体上分别负责什么

如果把后面的内容按能力块而不是按讲次编号来看，CS336 大体可以分成五层：

1. **表示与模型层**：tokenization、架构、训练基本块。
2. **系统与硬件层**：GPU、kernel、并行训练、推理。
3. **规模化决策层**：resource accounting、scaling laws、训练预算规划。
4. **数据与评估层**：评测基准、数据清洗、过滤、去重与预算分配。
5. **后训练与行为层**：SFT、RLHF、DPO、PPO、GRPO、RLVR。

这意味着课程不是“先讲模型，再顺手讲点系统”，而是把语言模型视为一个从表示到部署、从训练到对齐的完整工程对象。第一讲虽然只正式展开了 tokenization，但已经把后面所有主题的存在理由都说明了。



(a) 架构层：模型骨架与表示方式

(b) 系统层：推理阶段的 prefill / decode 分化

图 8: 第一讲虽然没有细讲所有模块，但已经把后续关键模块在整体图景中的位置预告出来。

2.4 为什么这门课会非常重：因为它把研究工程肌肉当成主要训练目标

第一讲对课程 logistics 的描述其实也很有信息量。它强调这是一门 5-unit 课程，作业量极大，包含 tokenizer、Transformer、训练循环、kernel、并行、scaling law、数据和 alignment 等多条线，而且几乎不给完整 scaffolding code，只提供测试与接口。这种设置不是故意为难人，而是课程相信：**研究工程能力本来就不是看出来的，而是做出来的**。

课程甚至非常明确地告诉学生，哪些人适合、哪些人不适合来上。适合的是那种“非得搞清楚东西怎么工作”的人；不适合的是只想快速追热点、只想在当前季度尽快出研究结果、或只想把现成模型套到自己应用域的人。这个界限其实很诚实：CS336 不承诺短期收益，它承诺的是长期能力建设。

“Executable lecture” 的教学意义

第一讲反复出现 `lecture_01.py` 的执行界面，说明课程讲义本身就是程序。这样做不是噱头，而是为了让学生把“概念、代码、结构、实验对象”放到同一空间里理解。这种安排非常符合课程强调的 building mindset。

2.5 本章小结

到这里，第一讲已经完成了第二层任务：它不只告诉你“为什么这门课存在”，还告诉你“你应该以什么方式来上这门课”。CS336 训练的不是对若干热点关键词的记忆，而是一种围绕资源预算做系统设计决策的能力。这种能力一旦形成，后面的架构、并行、推理、数据和对齐章节就不再是孤立知识点，而是同一个问题的不同侧面。

3 历史演化与开放性格局：这门课站在什么时间点上

3.1 从 Shannon 到 Transformer：语言建模问题是怎样形成的

第一讲还用一条历史时间线，把今天的大模型放回更长的研究脉络里。课程并没有把语言模型历史写成“忽然出现 GPT-4”的神话，而是明确指出几个关键阶段：

- pre-neural 时代，语言模型首先是统计建模问题，比如用语言模型衡量英语熵、用 n-gram 支持机器翻译和语音识别。
- neural ingredients 时代，神经语言模型、seq2seq、Adam、attention、Transformer、MoE 与 model parallelism 等关键模块逐渐被发明出来。
- early foundation model 时代，ELMo、BERT、T5 等模型证明了大规模预训练对下游任务的迁移价值。
- embracing scaling 时代，GPT-2、GPT-3、PaLM、Chinchilla 把“规模化”从一个经验方向变成了研究范式。

这条时间线的作用，不是让读者背论文，而是帮助我们理解：**今天大模型世界里的很多东西，其实都是旧组件在新规模条件下被重新组合后的结果**。课程因此特别强调，不要把现代语言模型理解成某个单点发明，而要把它看作表示、优化、并行、硬件与数据共同演化的产物。

3.2 为什么“扩规模”会改写整个研究范式

从 GPT-2 开始，到 GPT-3、PaLM、Chinchilla，再到后续的开放模型浪潮，研究问题的中心逐渐发生了变化。过去人们更常问“这个架构新不新”；而在 scaling 时代，更核心的问题往往是：

- 在固定 FLOPs 预算下，模型和数据应该如何配比？
- 哪些超参数在小规模调优后能稳定外推到大规模？
- 架构改动到底是在增加能力，还是在降低系统成本？
- 推理成本会不会反过来改变训练时的最优选择？

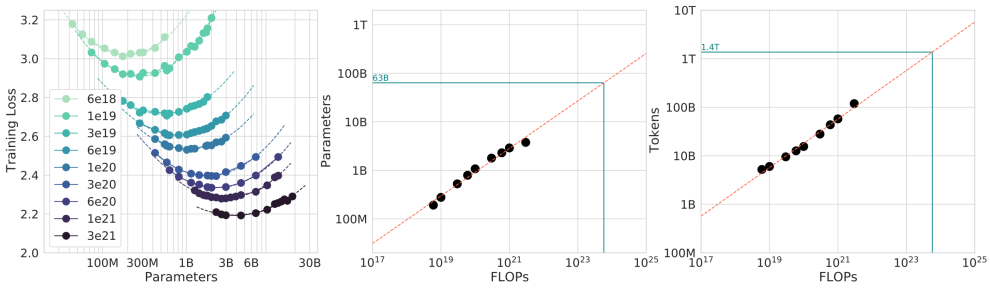


图 9: 课程后续会反复讨论的 scaling 视角：不是“盲目做大”，而是问在固定预算下怎么做得更优。第一讲已经埋下了这个问题。

也正因为此，第一讲才会那么早地把效率世界观抛出来。规模化不是简单把所有东西变大，而是迫使你重新思考每一个设计决策是否真的值得其资源成本。

3.3 开放性不是二元变量，而是一条连续谱

第一讲对“开放”的理解也很务实。课程并不把开放简化成开源/闭源二分，而是至少区分三层：

开放层级	典型特征
闭源模型	只有 API 可用，训练数据、权重、细节大多不可见。
开权重模型	权重可获得，通常会公开架构和部分训练细节，但数据、失败实验与完整 rationale 往往缺失。
更接近开放研究的模型	尽量公开权重、数据、实现与实验细节，但仍未必能覆盖全部工程背景与决策过程。

表 2: 课程对 openness 的理解是连续谱，而不是简单二分。

这点非常关键。因为很多学习者会误以为“有权重 = 足够理解系统”，但课程明确指出，就算有权重、有论文，你通常仍然看不到训练 rationale、失败实验、资源约束与关键工程折中。正因如此，真正可持续的做法不是迷信某个单一开放模型，而是学会从多个公开系统里抽取可泛化的共性。

3.4 “今天的 frontier models”是课程时点的快照，而不是永恒结论

第一讲还罗列了课程录制时点的 frontier models，例如 OpenAI、Anthropic、xAI、Google、Meta、DeepSeek、Alibaba、Tencent 等系统。这里最值得注意的，不是名单本身，而是课程用它来告诉学生：**前沿世界变化极快，所以你不能把“记住当前最强模型是谁”当成核心能力**。真正稳的能力，是看懂一个系统的资源结构、训练范式、推理特性和后训练路线。

如何正确阅读“today's frontier models”这页

这页的价值主要是帮助学生定位课程所处时代背景，而不是给出永恒模型排行榜。真正能长期保值的，是课程后面训练出的分析框架，而不是这一时点的品牌名录。

3.5 本章小结

历史时间线与开放性讨论，帮助我们给第一讲再加上一层上下文：CS336 既不是在否认 scaling 时代的现实，也不是在浪漫化“人人都该重训 GPT-4”。它是在承认工业现实的同时，重新定义普通研究者与工程师还能掌控什么能力。第一讲的真正目的，是在这样一个现实世界里，为后续学习划定合理而有力量的起点。

4 Tokenization：整条流水线里的第一个具体设计决策

4.1 Tokenizer 的职责：在字符串与离散序列之间搭桥

在完成课程世界观的铺垫之后，第一讲终于落到第一个具体技术模块：tokenization。课程给出的定义很朴素：tokenizer 负责在字符串和整数序列之间做双向转换。表面看，这只是一个前处理问题；实际上，它是整条语言模型流水线的入口。

如果没有 tokenization，今天主流自回归语言模型就没有稳定的离散输入对象；而一旦采用某种 tokenization，后面的 embedding 维度、序列长度、attention 成本、KV cache 容量、数据利用率甚至推理速度都会连锁变化。所以课程把 tokenizer 放在第一讲最后，不是因为它简单，而是因为它是第一个真正把“表示能力”和“资源成本”绑到一起的设计点。

4.2 为什么这门课从 BPE 开始，而不是直接追求 tokenizer-free 的优雅

课程明确选择从 Byte-Pair Encoding 开始，这个选择背后其实是工程现实。理论上，直接用 bytes 处理文本非常统一，也更少人工设计；但在当前主流架构与算力条件下，纯 byte 方案通常会导致更长序列，从而显著抬高后续计算与内存开销。相比之下，BPE 通过把高频片段合并为子词，可以在表达能力与计算效率之间取得一个更务实的折中。

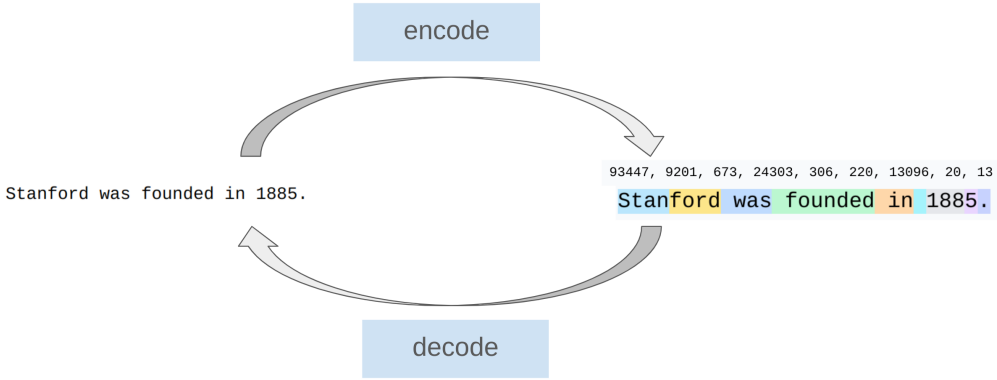


图 10: 课程用一个直观例子说明 tokenizer 的核心直觉：把字符串拆成出现频繁、对建模有利的离散片段。

课程并没有把 BPE 神化成“语言本质的最佳表示”。恰恰相反，第一讲的语气非常节制：它把 BPE 视作当前工程条件下的合理起点，而不是终极答案。这种态度和前面对效率世界观的强调完全一致。

4.3 视频里的 tokenization 段落，其实已经在预演后面整门课

从视频画面看，讲者在 `lecture_01.py` 的执行界面里，把 tokenization 放在“Goal: get a basic version of the full pipeline working”之下，和 architecture、training 紧密相连。这一点很重要：课程不是把 tokenizer 当成独立算法小节，而是把它当成 full pipeline 的第一块拼图。

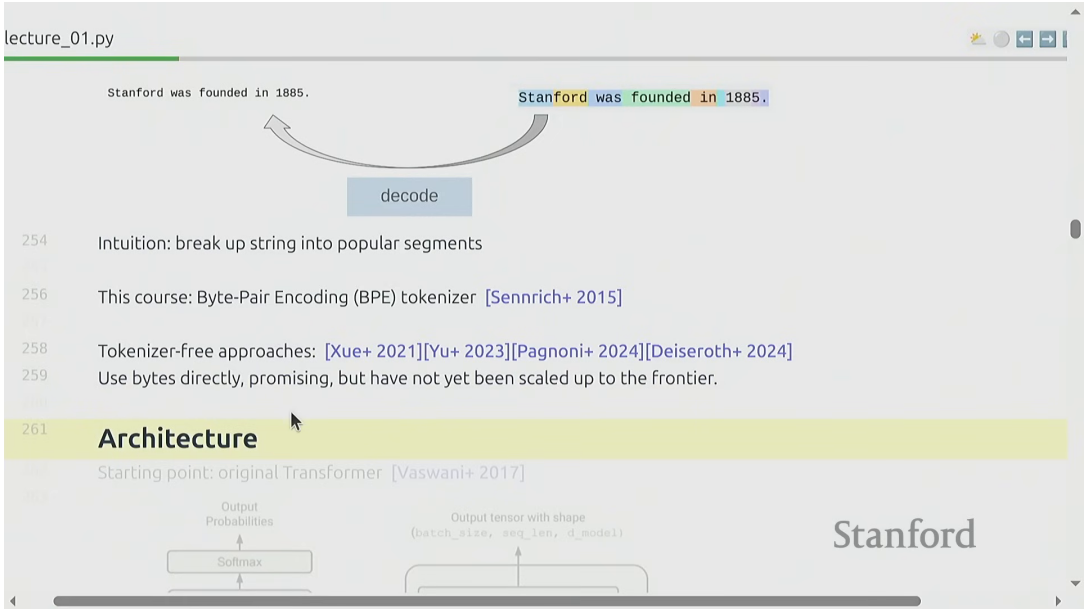


图 11: 课程视频中的 tokenization 页面：BPE 被放在完整 pipeline 的起点位置，而不是孤立预处理步骤。[3]

这种编排本身已经说明了它的重要性。你一旦选了某种 tokenization，就已经在替后面的 architecture、training 与 inference 做一部分选择。用课程自己的话说，这些选择最终都要回到“给定预算下如何做得最好”。

4.4 Tokenization 不是孤立预处理，它会改变后续所有模块

从系统视角看，tokenization 至少会影响四件事：

- **序列长度**：token 更细，序列更长，attention 与 KV cache 成本随之上升。
- **词表规模**：词表越大，embedding 与 softmax 层的参数规模越大。
- **数据利用率**：切分策略不同，模型在同样语料上能学到的模式也不同。
- **推理体验**：token 粒度会影响输出节奏、缓存管理和吞吐表现。

所以，tokenization 真正体现的是第一讲前半段一直在强调的那件事：**表示方式本身也是资源分配问题**。这也是为什么课程后面会继续把 tokenization 和 architecture、scaling、推理放在同一逻辑链上讨论。

为什么 tokenizer-free 路线仍值得关注

课程虽然没有在第一讲展开 tokenizer-free 模型，但已经明确提到 bytes 直接建模等路线“很有希望，只是还没有在 frontier 规模上完全跑通”。这句话的潜台词是：tokenization 不是永恒真理，而是当前代模型系统的现实折中。

4.5 从第一讲到第二讲：为什么下一步自然是 PyTorch 与资源核算

第一讲最后一句话直接预告了下一讲：PyTorch building blocks 与 resource accounting。这个转场非常自然，因为到此为止，课程已经完成了两个动作：

[3] 视频画面时间区间：00:28:58–00:29:18。该图来自课程视频中的屏幕画面。

1. 先建立“效率驱动设计”的世界观。
2. 再选择 tokenization 作为第一个具体设计问题。

下一步就该问：**既然我们要真的开始 build，那么这些对象在张量、显存、FLOPs 和训练循环里究竟怎么落地？**换句话说，第一讲负责给出问题框架，第二讲才开始把这个框架压缩成可核算的工程对象。

4.6 本章小结

第一讲对 tokenization 的处理虽然后置，但意义并不轻。它表明“从零构建语言模型”不是先把模型层背下来，而是先承认所有后续模块都要建立在一种离散表示方案之上。课程选择 BPE，不是因为它优雅，而是因为在当前主流算力和架构条件下，它代表了一个对整体系统比较友好的起点。

5 总结与延伸

把整场第 1 讲压缩起来，可以得到四个最重要的 takeaway。

第一，**这门课真正想解决的，是抽象层不断上移后研究者与底层系统脱节的问题**。课程并不反对高层抽象，而是提醒你：如果要做更深入的研究与工程决策，就必须知道抽象是如何泄漏的。

第二，**frontier model 的工业化现实，并没有让“从零理解”失去意义，反而让它更需要被重新定义**。你当然不必把 GPT-4 从头训练一遍，但你必须能区分哪些机制、思维方式与成本模型在不同规模下仍然成立。

第三，**第一讲已经把整门课的统一主线讲清楚了：accuracy 不是孤立模型结构的结果，而是 efficiency 与 resources 联合作用的产物**。这一点会在后续 GPU、kernel、parallelism、scaling law、inference、data 和 alignment 章节里不断回响。

第四，**tokenization 不是一个可有可无的小预处理模块，而是整条流水线里第一个正式的系统设计决策**。它把表示方式、训练成本和推理成本绑在了一起，因此非常适合作为“从零构建”的技术起点。

我对第 1 讲的压缩结论

这不是一堂“语言模型概览课”，而是一堂“如何正确看待语言模型系统”的世界观课程。它最核心的贡献，不是给出若干定义，而是把后面 16 讲统一进了一个资源受限的系统设计框架。

从后续学习路径看，这一讲之后最自然的三个深化方向是：

- 把“效率驱动设计”的世界观落到可量化对象上，也就是第 2 讲的资源核算与 PyTorch 基础。
- 把 tokenization 进一步放回训练与推理链路，理解它如何影响后面的 attention、KV cache 与 scaling。
- 在整门课结束后，再回头重读这堂导论，你会更清楚它为什么花了大量时间谈工业化、开放性和课程设计，而不是一开始就写公式。